

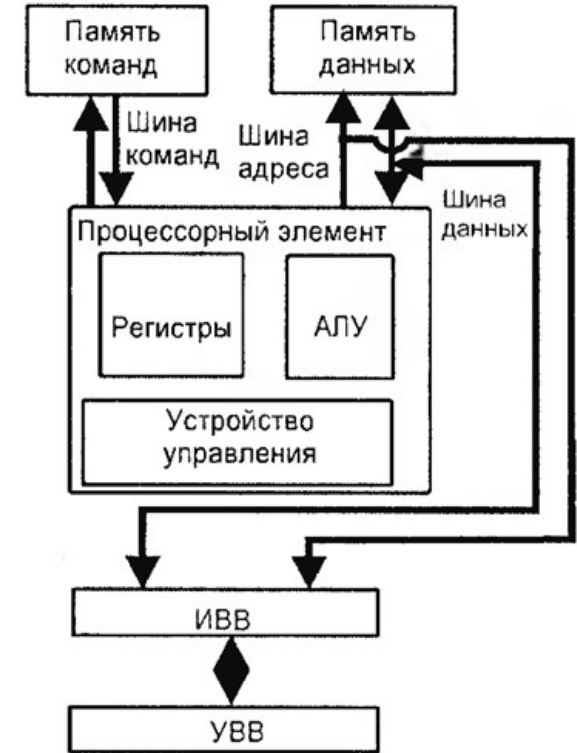
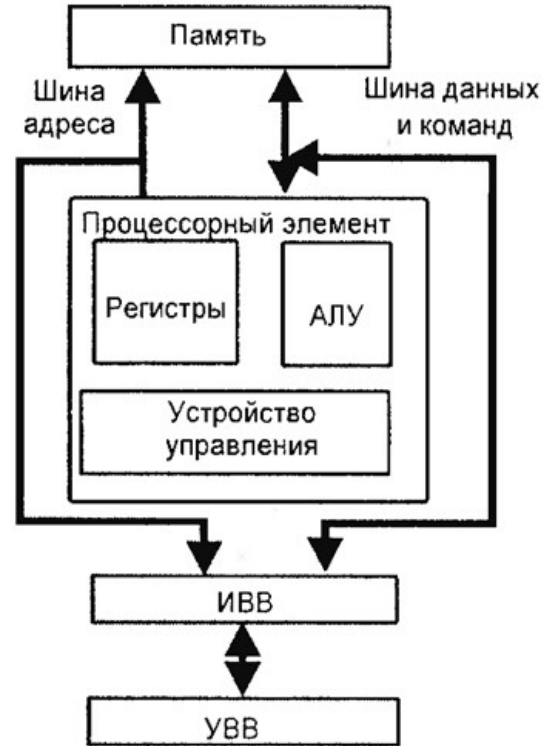
Занятие 0x01

Языки программирования низкого уровня

Введение

Архитектура ЭВМ

- Архитектура фон Неймана
- Гарвардская архитектура



Архитектура фон Неймана

- Использование двоичной системы счисления в вычислительных машинах
- Программное управление ЭВМ
- Память компьютера используется не только для хранения данных, но и программ
- Ячейки памяти ЭВМ имеют адреса, которые последовательно пронумерованы
- Возможность условного перехода в процессе выполнения программы

Гарвардская архитектура

- Хранилище инструкций и хранилище данных представляют собой разные физические устройства
- Канал инструкций и канал данных так же физически разделены

Микропроцессор

Центральный блок ПК, предназначенный для управления работой всех блоков машины и для преобразования информации

- Состоит из следующих схем:
 - Арифметико-логическое устройство (АЛУ)
 - Внутренние регистры
 - Устройство управления (УУ)
 - Интерфейс микропроцессора

Системная шина

Основная интерфейсная система ПК

- Включает в себя:
 - **шину данных**, содержащую провода и схемы сопряжения для параллельной передачи всех разрядов числового кода операнда
 - **шину адреса**, содержащую провода и схемы сопряжения для параллельной передачи всех разрядов числового кода адреса ячейки ОП или порта ввода-вывода внешнего устройства
 - **шину управления**, содержащую провода и схемы сопряжения для передачи управляющих сигналов во все блоки ПК
 - **шину питания**, имеющую провода и схемы сопряжения для подключения блоков ПК к системе электропитания
-

Виды памяти

- **Основная память (ОП)** – предназначена для хранения и оперативного обмена информацией с другими блоками ПК. ОП состоит из ПЗУ и ОЗУ. В последнее время в состав ОП входят также КЭШ- и FLASH-память
 - ПЗУ – служит для хранения постоянной программной и справочной информации, позволяет оперативно только считывать хранящуюся в нем информацию (изменить информацию в ПЗУ нельзя).
 - ОЗУ – предназначено для оперативной записи, хранения и считывания информации (программ и данных), непосредственно участвующей в информационно-вычислительном процессе, выполняемом ПК в текущий период времени.
- **Внешняя память** – используется для долговременного хранения любой информации, которая может когда-либо потребоваться. Во внешней памяти хранится все программное обеспечение ПК.

Устройство ОЗУ

- Нумерованный набор ячеек (байт) для хранения информации
- Каждая ячейка имеет номер — адрес
- Доступ только к целой ячейке, а не к отдельным битам

Принципы работы ЦП

- Для временного хранения и выполнения операций используются **регистры**
- В общем случае принцип работы следующий:
 - Считать инструкцию из ОЗУ
 - Выполнить инструкцию, например:
 - Считать значения из ОЗУ в регистры
 - Выполнить операции
 - Записать значение регистра с результатом в ОЗУ
 - Перейти к следующему адресу с инструкцией в ОЗУ

Виды процессоров

- **CISC** (complex instruction set computing)
 - Большое число различных инструкций
- **RISC** (reduced instruction set computing)
 - Сокращенное число простых инструкций
- **MISP** (minimal instruction set computing)
 - Максимально сокращенное число простых инструкций
- **VLIW** (very long instruction word)
 - Инструкции с параллельным выполнением

Ассемблер

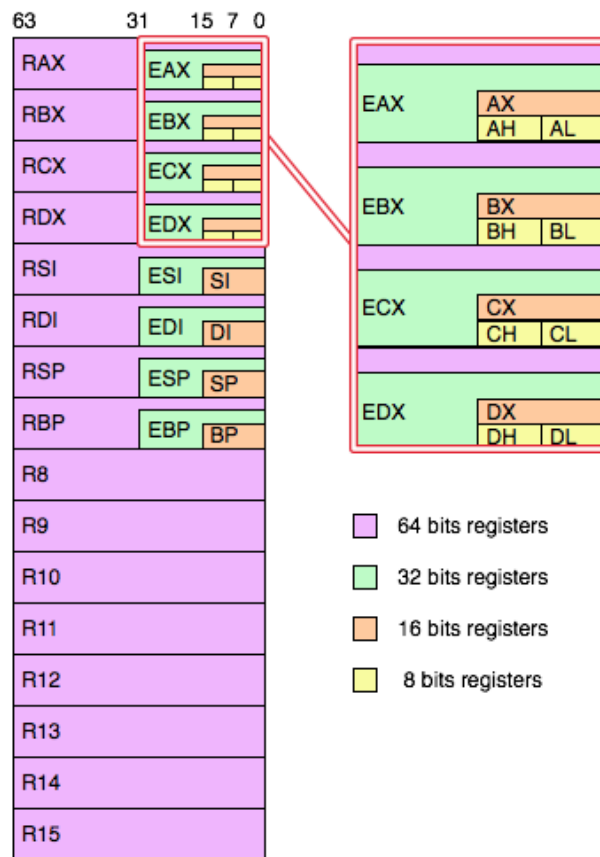
- Процессор «понимает» инструкции только в виде чисел
- Числа сложно воспринимаются человеком
- Для удобства программирования придумали ассемблер – **символическую запись инструкций процессора**
- Существуют различные языки ассемблера, отличающиеся синтаксисом (Intel Asm, AT&T и т.п.)
- Компилятор ассемблера – переводит символы в числа, соответствующие инструкциям

Регистры общего назначения

Size (in Bits)

64	32	16	8
RAX	EAX	AX	AH/AL
RBX	EBX	BX	BH/BL
RCX	ECX	CX	CH/CL
RDX	EDX	DX	DH/DL
RDI	EDI	DI	DIL
RSI	ESI	SI	SIL
RBP	EBP	BP	BPL
RSP	ESP	SP	SPL
R8~R15	R8D~R15D	R8W~R15W	R8L~R15L

General purpose registers



Базовые инструкции

- **MOV** оп1, оп2
 - Соответствует: $\text{оп1} = \text{оп2}$
- **ADD** оп1, оп2
 - Соответствует: $\text{оп1} = \text{оп1} + \text{оп2}$
- **SUB** оп1, оп2
 - Соответствует: $\text{оп1} = \text{оп1} - \text{оп2}$

Базовые инструкции

- **MUL** оп1
 - Соответствует: $RDX:RAX = RAX * оп1$
- **DIV** оп1
 - Соответствует:
 - $RAX = RDX:RAX / оп1$
 - $RDX = RDX:RAX \% оп1$

Пример кода

Ассемблер:

```
mov rax, 10  
mov rbx, 20  
mov rcx, rax  
add rax, rbx  
sub rbx, rcx
```

Псевдокод:

```
a = 10  
b = 20  
c = a  
a = a + b  
b = b - c
```

Пример представления

00000000	B80A000000	mov eax,0xa
00000005	BB14000000	mov ebx,0x14
0000000A	4889C1	mov rcx,rcx
0000000D	4801D8	add rax,rbx
00000010	4829CB	sub rbx,rcx